Автоматическая оценка лексической сложности англоязычных песен

Финальный проект по программе
«Компьютерная лингвистика»
НИУ ВШЭ

София Поповичева

```
show_song_info(row, cefr_dict):
print(f"ARTIST: {row['artist']}")
print(f"SONG:   {row['title_en']}")
print(f"CEFR top10: {row.get('cefr_top10_mean_level')}
print("\nEnglish\n")
print(row.get("original_text", ""))
print("\nРусский\n")
print(row.get("translation_text", ""))

lemmas = row.get("lemmas", [])
hard_words = hard_vocab_from_lemmas(lemmas, cefr_dict,

print("\nПолезная лексика")
if len(hard_words) == 0:
    print("Слов уровня B1-B2 не нашлось.")
else:
    for w, c in hard_words:
        print(f"{w} - {c}")



ist_query = input(" Введи артиста (например, Eminem): ")
```

Цели и Задачи

Цель проекта:

Разработать модель автоматической оценки лексической сложности англоязычных песен на основе статистических и словарных характеристик текста.

Основные задачи:

- собрать корпус текстов песен
- выполнить предобработку и лемматизацию
- извлечь формальные признаки текста
- рассчитать показатели сложности на основе CEFR
- объединить признаки и провести кластеризацию
- реализовать демонстрационный модуль анализа песни

Сбор данных

```python
def scrape_artist(artist_name, artist_url):
    """ Парсит все песни одного ариста. Возвращает список словарей, где
    Каждый словарь содержит:
    - artist
    - song_url
    - title_en
    - title_ru
    - original_text
    - translation_text"""

    song_urls = get_song_links(artist_url)
    rows = []

    for i, song_url in enumerate(song_urls, start=1):
        time.sleep(SLEEP_BETWEEN_REQUESTS)

        try:
            title_en, title_ru, original_text, translation_text = parse_

            rows.append({
                "artist": artist_name,
                "song_url": song_url,
                "title_en": title_en,
                "title_ru": title_ru,
                "original_text": original_text,
                "translation_text": translation_text
            })

            print(f"[{artist_name}] {i}/{len(song_urls)} OK - {title_en

        except Exception as e:
            print(f"[{artist_name}] {i}/{len(song_urls)} ERROR - {song_

    return rows
```

Тексты песен были автоматически получены с сайта Amalgama с помощью парсинга:

- получение списка песен для каждого исполнителя
- переход по ссылкам на страницы песен
- извлечение английского текста и перевода
- сохранение данных в табличный формат

```
Начинаю парсить: Adele
Ссылка: https://www.amalgama-lab.com/songs/a/adele/
[Adele] 1/63 OK - All I Ask
[Adele] 2/63 OK - All Night Parking
[Adele] 3/63 OK - Best for Last
[Adele] 4/63 OK - Black & Gold*
[Adele] 5/63 OK - Can I Get It
[Adele] 6/63 OK - Can't Let Go
[Adele] 7/63 OK - Chasing Pavements
[Adele] 8/63 OK - Cold Shoulder
[Adele] 9/63 OK - Crazy for You
[Adele] 10/63 OK - Cry Your Heart Out
[Adele] 11/63 OK - Daydreamer
[Adele] 12/63 OK - Don't You Remember
[Adele] 13/63 OK - Easy on Me
[Adele] 14/63 OK - Fastlove*
[Adele] 15/63 OK - First Love
[Adele] 16/63 OK - Fool That I Am
[Adele] 17/63 OK - He Won't Go
[Adele] 18/63 OK - Hello
[Adele] 19/63 OK - Hiding My Heart*
[Adele] 20/63 OK - Hold On
[Adele] 21/63 OK - Hometown Glory
...
==========
Сбор завершён!
Всего строк (песен): 1389
Колонки: ['artist', 'song_url', 'title_en', 'title_ru',
```

Output is truncated. View as a scrollable element or open in a text edi

	artist	song_url
0	Adele	https://www.amalgama-lab
1	Adele	https://www.amalgama-lab
2	Adele	https://www.amalgama-lab

Обработка текста

```python
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string

lemmatizer = WordNetLemmatizer()
stop_w = set(stopwords.words('english'))

def lemmatize_song(text):
    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word.isalpha()]
    tokens = [word for word in tokens if word not in stop_w]
    lemmas = [lemmatizer.lemmatize(word) for word in tokens]
    return lemmas
```

```python
df["lemmas"] = df["original_tex
```

lemmas
['verse', 'leave', 'heart', 'door', 'wo', 's
['know', 'got', 'thing', 'want', 'spend',
['wait', 'see', 'heart', 'sleeve', 'day', 'e

После сбора корпуса тексты были приведены к анализируемому виду

- токенизация текста
- приведение слов к нижнему регистру
- удаление стоп-слов
- фильтрация неалфавитных токенов
- лемматизация

Статистические признаки текста

Для оценки сложности были рассчитаны формальные характеристики текста:

- **n_tokens** — общее количество слов
- **TTR (type-token ratio)** — разнообразие лексики
- **avg_word_len** — средняя длина слова
- **long_word_ratio** — доля длинных слов

```
df["n_tokens"] = df["lemmas"].apply(len) # общее кол-во слов в песн
df["n_unique"] = df["lemmas"].apply(lambda x: len(set(x))) # уникал
df["ttr"] = df["n_unique"] / df["n_tokens"] # Type token ratio -- у
df["avg_word_len"] = df["lemmas"].apply(lambda x: sum(len(w) for w
```

```
df[["artist", "title_en", "n_tokens", "n_unique", "ttr", "avg_word_
```

	artist	title_en	# n_toker
0	Adele	All I Ask	
1	Adele	All Night Parking	
2	Adele	Best for Last	
3	Adele	Black & Gold*	
4	Adele	Can I Get It	
5	Adele	Can't Let Go	

Оценка сложности по CEFR

```python
level_nums = [level_order[lvl] for lvl in levels] # перевожу уровни в числа, где A1
mean_num = sum(level_nums) / len(level_nums)    # среднее арифметическое по словам (е

rounded = int(round(mean_num)) # округляю средний уровень до ближайшего (1..6)
inv_level_order = {v: k for k, v in level_order.items()} # и обратно перевожу число в
mean_level = inv_level_order.get(rounded) # и округляю к ближайшему целому числу

level_nums_sorted = sorted(level_nums)

# p90 уровень "сложного хвоста" (примерно самые сложные 10% слов)
idx = int(0.9 * (len(level_nums_sorted) - 1)) # 90 перцентиль и среднее по самым слож
p90_num = level_nums_sorted[idx]
p90_level = inv_level_order.get(p90_num)

# top10_mean средний уровень по самым сложным 10% словам
k = max(1, int(len(level_nums_sorted) * 0.10))  # хотя бы 1 слово
top10 = level_nums_sorted[-k:]
top10_mean_num = sum(top10) / len(top10)
top10_mean_level = inv_level_order.get(int(round(top10_mean_num)))

total = len(levels) # Общее число известных CEFR-слов в песне
shares = {f"share_{lvl}": levels.count(lvl) / total for lvl in level_order.keys()} #

return {
    "cefr_coverage": coverage,
    "cefr_mean_level_num": mean_num,
```

Для каждой песни рассчитывались:

- доля слов, найденных в словаре (coverage)
- средний уровень лексики
- уровень 90-го перцентиля
- средний уровень самых сложных 10% слов

Для более интерпретируемой оценки сложности был использован CEFR-словарь (A1–B2).

```python
# загружаю подготовленный датасет с лексикой ра
cefr = pd.read_csv('word_list_cefr.csv', sep=";

cefr = cefr[["headword", "CEFR"]].copy() # оста
cefr.columns = ["word", "level"]
cefr["word"] = cefr["word"].str.lower()
cefr.head()
```

	word	level
0	a	A1
1	a.m.	A1
2	a.m.	A1
3	abandon	B1
4	abandoned	B2

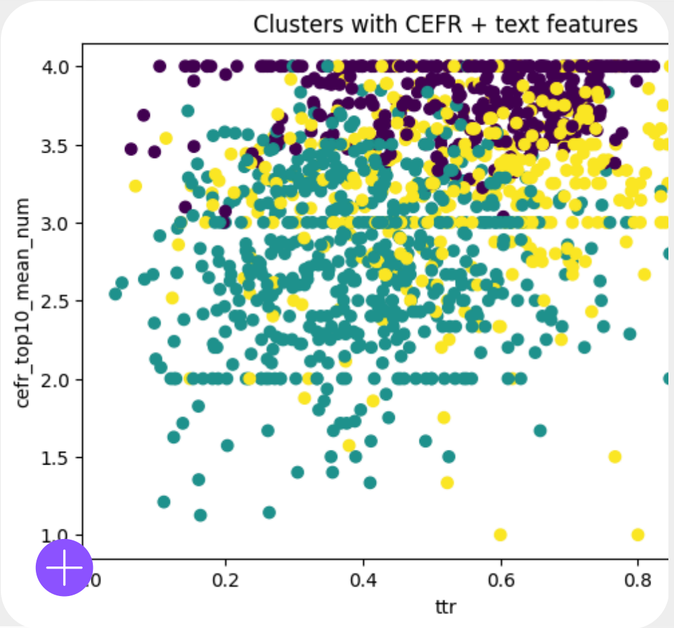
* *Среднее значение часто занижает уровень текста из-за высокой доли частотных слов, поэтому дополнительно анализировался «сложный хвост» распределения.*

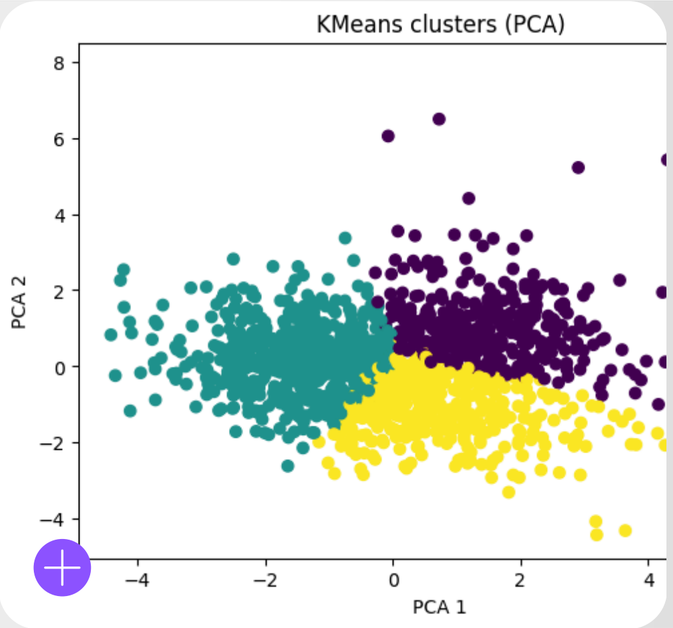
Объединённая модель и кластеризация

```python
feature_cols = [
    "n_tokens",                 # длина текста
    "ttr",                      # разнообразие лексики (u
    "avg_word_len",
    "long_word_ratio",          # доля длинных слов
    "cefr_coverage",            # доля лемм, найденных в
    "cefr_top10_mean_num",      # средний уровень самых с
    "share_B2"                  # доля слов уровня B2 сре
]
if "long_word_ratio" not in df.columns:
    df["long_word_ratio"] = df["lemmas"].apply(
        lambda xs: sum(len(w) >= 6 for w in xs) / len(x
    )

df_clust = df.copy()  # копия датафрейма для кластериза
df_clust = df_clust.dropna(subset=feature_cols)  # выбр
X = df_clust[feature_cols].values  # массив признаков

print(df.columns.tolist()) # вышло, что одного признак
```





Были объединены:

- статистические признаки текста
- CEFR-показатели

Для **группировки** песен использовался алгоритм **KMeans** (3 кластера). Полученные группы интерпретируются как:

- **easy**
- **medium**
- **hard**

PCA используется для проекции **многомерных признаков в двумерное пространство**, что позволяет наглядно отобразить структуру кластеров.

```python
artist_query = input(" Введи артиста (например, Eminem): ")
df_artist = find_artist(df, artist_query)

if len(df_artist) == 0:
    print("Нет такого артиста. Попробуй другое написание.")
else:
    titles = sorted(df_artist["title_en"].dropna().unique().tolist())
    print(f"\nНайдено {len(titles)} песен. Примеры:")
    for t in titles[:30]:
        print("-", t)

    song_query = input("\nВведи название песни: ")
    df_song = find_song(df_artist, song_query)

    if len(df_song) == 0:
        print("Нет такой песни. Попробуй по-другому.")
    else:
        df_song = df_song.reset_index(drop=True)
```

Пользователь может:

- ввести имя исполнителя
- выбрать песню
- получить текст на английском и русском
- увидеть наиболее сложную лексику (уровни B1–B2)

В рамках проекта реализован демонстрационный модуль анализа песни.
Это позволяет использовать модель для отбора песен по уровню сложности и изучения продвинутой лексики.

Демонстрационный модуль *

*Ограничения и Выводы

Ограничения

- CEFR-словарь покрывает только уровни A1–B2
- не учитывается синтаксическая сложность
- не анализируются устойчивые выражения
- кластеризация не даёт абсолютных уровней, а лишь относительное разделение

Выводы

- разработана модель оценки лексической сложности песен
- предложена метрика анализа «сложного хвоста» лексики
- песни могут быть сгруппированы по уровням сложности
- модель может использоваться для отбора учебных материалов

Спасибо за внимание